SOFTWARE REQUIREMENTS SPECIFICATION

for

ProjNaN

An Educational Collaborative Project Management and Contribution Verification Platform

Version 1.0

Prepared by:

Vanshaj Sharma (Roll No: 1024160114)
Aryan Thakkar (Roll No: 1024160100)

Group: 3P14, BE Third Year, COSE

Submitted to:

Dr. Anushka
Assistant Professor
Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Course: UCS 503 – Software Engineering

October 15, 2026

Contents

1 Introduction **3**
- 1.1 Purpose . . . 3
- 1.2 Intended Audience and Reading Suggestions . . . 3
- 1.3 Project Scope . . . 3

2 Overall Description **5**
- 2.1 Product Perspective . . . 5
- 2.2 User Classes and Characteristics . . . 5
- 2.3 Product Functions . . . 6
- 2.4 Operating Environment . . . 7
- 2.5 Design . . . 7

3 System Features **10**
- 3.1 Description and Priority . . . 10
- 3.2 Functional Requirements . . . 10
  - 3.2.1 MOD-01: User Authentication, Session Management, and RBAC . . . 10
  - 3.2.2 MOD-02: Academic Class Scoping and Team Formation . . . 11
  - 3.2.3 MOD-03: Project Workspaces and Milestone Tracking . . . 11
  - 3.2.4 MOD-04: Task Management and 4-Stage State Lifecycle . . . 11
  - 3.2.5 MOD-05: Evidence Submission and Rule-Based Heuristic Evaluation . . . 12
  - 3.2.6 MOD-06: Faculty Task Verification and Confirmation . . . 12
  - 3.2.7 MOD-07: External GitHub Telemetry Synchronization . . . 13
  - 3.2.8 MOD-08: Idempotent Contribution Ledger and Team Analytics . . . 13
  - 3.2.9 MOD-09: Gamification, Weekly Arena, and Leaderboard . . . 14
  - 3.2.10 MOD-10: Pedagogical Feedback and 1–5 Star Reviews . . . 14
  - 3.2.11 MOD-11: Persistent Notifications and Project Channel Discussion . . . 14

4 Other Nonfunctional Requirements **15**
- 4.1 Performance Requirements . . . 15
- 4.2 Security Requirements . . . 15
- 4.3 Software Quality Attributes . . . 16
- 4.4 Business Rules . . . 16

5 Other Requirements **17**
- 5.1 Maintenance Requirements . . . 17
- 5.2 Future Roadmap . . . 17

1 Introduction

1.1 Purpose

In university computer science courses, Project-Based Learning is a core teaching method. Students collaborate in small teams to design and build software systems over a semester. However, evaluating collaborative student projects is difficult for course instructors. Traditional grading relies on periodic milestone presentations, written status reports, or final compressed code archives. These methods give teachers little direct visibility into individual participation, code authorship regularity, or task authenticity. Often, a few active students do most of the work while others contribute very little (social loafing or free-riding).

ProjNaN is an educational project management and contribution verification web platform built to solve this problem. ProjNaN connects classroom administration (teachers, classes, student teams) with modern version control activity from GitHub (commits, pull requests, and issues). By combining interactive Kanban task boards with repository tracking and an internal rule-based verification engine, ProjNaN provides an authentic, transparent record of each student's technical contributions.

1.2 Intended Audience and Reading Suggestions

This Software Requirements Specification (SRS) is intended for course instructors, student developers, academic evaluators, software architects, and quality assurance testers. It describes the complete functional requirements, system behavior, user roles, technical environment, and nonfunctional qualities of ProjNaN.

Readers interested in the overall concept and user roles should review Chapters 1 and 2. Developers and testers should examine the detailed functional requirements in Chapter 3 and nonfunctional constraints in Chapter 4. Maintenance and future extensions are outlined in Chapter 5.

1.3 Project Scope

ProjNaN provides an academic workspace tailored for university courses. Instructors create academic classes, generate unique join codes, and supervise student teams. Students enroll in classes, form teams, organize tasks on a Kanban board, and link their team's remote GitHub repository.

When a student finishes an assigned task, they submit completion evidence such as pull request citations, commit hashes, and descriptive notes. An internal deterministic rule-based evaluation engine checks the submitted evidence against synchronized repository records and assigns an assistive confidence score ($0.20 - 0.98$). Course instructors hold final verification authority: approving a task awards gamified Experience Points (XP) to the student and records an immutable entry in the project contribution ledger, while rejecting a task returns it to the student with feedback for revision.

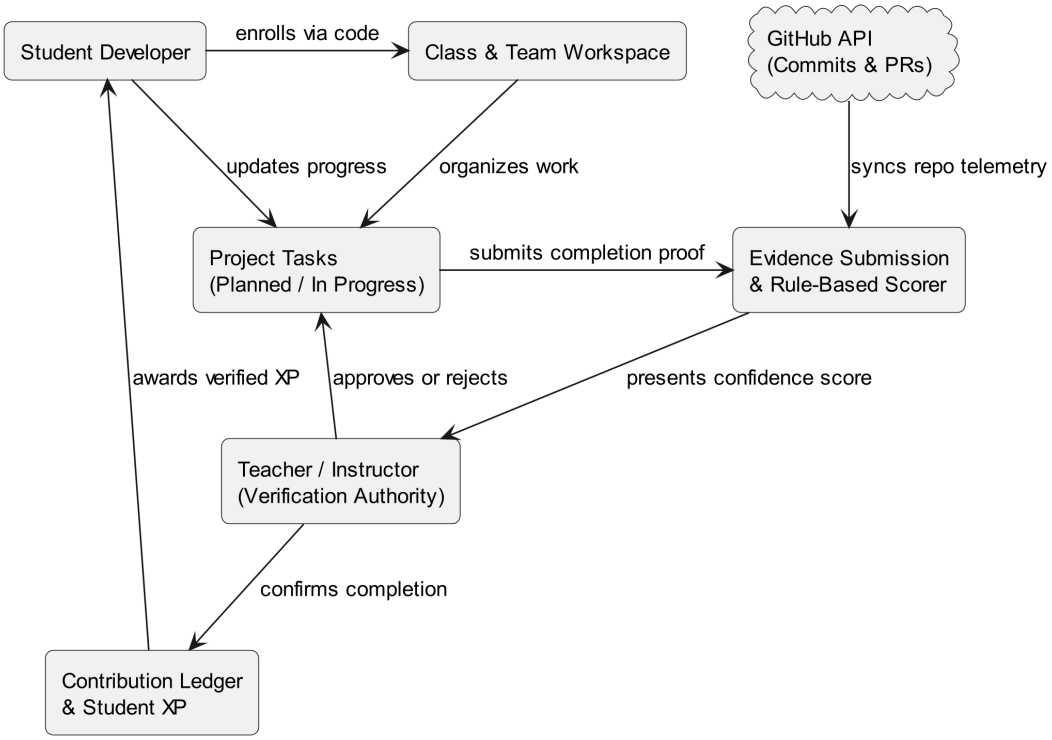


Figure 1.1: Overall Workflow of the ProjNaN Platform

Figure 1.1 illustrates the overall workflow of ProjNaN across two interacting cycles:

- **Student Cycle**: Students enroll in classes using unique join codes, form collaborative teams, and create discrete tasks. While implementing features, they commit code to GitHub, submit completion evidence on ProjNaN, and receive verified XP upon instructor approval.
- **Teacher Cycle**: Teachers establish course sections, supervise project teams, inspect student completion evidence alongside rule-based confidence scores, make authoritative verification decisions, and review relative effort distribution across team members.

2 Overall Description

2.1 Product Perspective

ProjNaN is an independent, self-contained web platform designed to replace manual, unverified project grading. Commercial project management tools (such as Jira or Trello) are built for enterprise software teams; they lack academic course hierarchies, peer accountability mechanisms, and instructor verification gates. Conversely, standard learning management systems track grades but have no connection to version control systems where students actually write code.

ProjNaN bridges this gap. It acts as an educational project management system that combines course rosters and team workspaces with automated GitHub telemetry ingestion, rule-based verification pre-scoring, and an idempotent contribution accounting ledger.

2.2 User Classes and Characteristics

ProjNaN serves two primary in-application user roles:

- **Course Instructor (Teacher)**: Academic faculty members who create classes, distribute join codes, set milestones, inspect student evidence, approve or reject task verifications, submit qualitative evaluations (1–5 star reviews), and review team contribution analytics.
- **Student Developer (Student)**: Enrolled undergraduate students who join classes using class codes, organize into project teams, create and claim Kanban tasks, submit technical evidence, complete weekly arena coding challenges, and track their earned XP and velocity.

In addition, two specialized operational responsibilities exist within the system:

- **Student Team Leader**: An enrolled student designated as team leader who holds the additional responsibility of linking the team's remote GitHub repository and coordinating milestone target dates. This is an operational responsibility, not a separate application role.
- **Infrastructure Administrator**: A DevOps or system administrator operating strictly at the host server infrastructure layer (server provisioning, SQLite backups, environment variables). This actor does not participate as an in-app user.

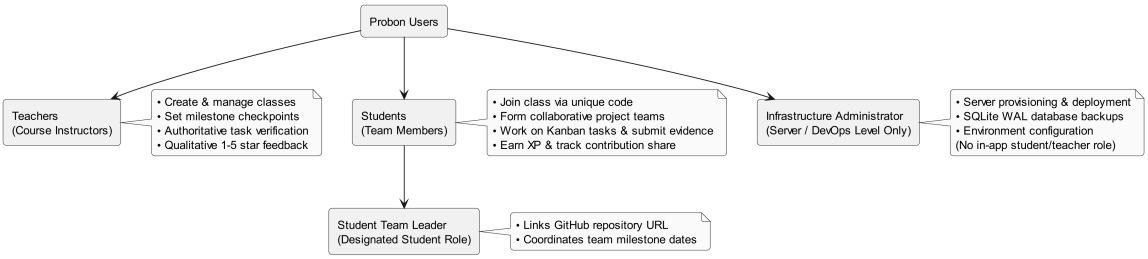


Figure 2.1: ProjNaN User Hierarchy and Operational Responsibilities

2.3 Product Functions

The primary functions of ProjNaN include:

1. **Authentication and Access Control**: Student and teacher registration, secure password hashing with bcrypt, stateless session management using JSON Web Tokens (JWT), email verification tokens, and password reset workflows.
2. **Academic Hierarchy and Team Scoping**: Teacher-managed class sections, unique collision-resistant join codes, student self-enrollment, team formation, and roster visibility.
3. **Project Workspaces and Milestones**: Dedicated team project spaces, milestone target dates, and automated dynamic progress percentage calculations.
4. **4-Stage Task State Machine**: Structured task lifecycle transitions (`Planned` → `In Progress` → `Verification Pending` → `Completed`) preventing arbitrary student self-completion.
5. **Heuristic Evidence Evaluation**: Rule-based pattern matching and repository cross-referencing that generates an assistive confidence score ($0.20 - 0.98$) for instructor review.
6. **Authoritative Faculty Verification**: Instructor approval or rejection of submitted tasks, ensuring only verified work earns gamified XP and contribution points.
7. **GitHub Telemetry Synchronization**: Background ingestion of remote Git commits, pull requests, and issues from the GitHub REST API v3 into local storage.
8. **Idempotent Contribution Ledger and Team Analytics**: Unique contribution keys preventing duplicate credit, individual effort share calculations (P_i), and rolling 14-day velocity sparklines.
9. **Gamification and Weekly Arena**: Difficulty-tiered coding challenges, achievement badges, and student rank progression from Novice to Master.
10. **Pedagogical Feedback**: Structured 1–5 star instructor ratings and qualitative reviews.
11. **Persistent Notifications and Channels**: Event-driven alert badges and project discussion channels.

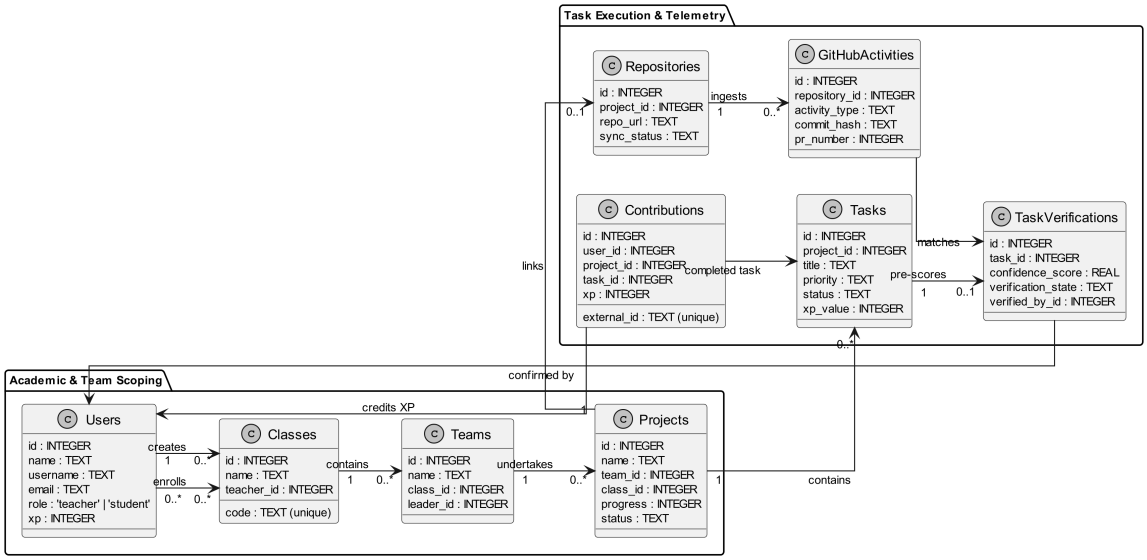


Figure 2.2: ProjNaN Data Overview and Core Entity Relationships

Figure 2.2 presents the relational data overview. A `User` creates or enrolls in `Classes`. A `Class` contains collaborative `Teams`, each undertaking a `Project`. Projects contain `Tasks` that associate with `TaskVerifications`. Linked `Repositories` ingest remote `GitHubActivities`, while verified task completions and commits generate immutable records in the `Contributions` ledger.

2.4 Operating Environment

ProjNaN is designed to run efficiently on standard computer laboratory infrastructure without requiring proprietary enterprise hardware:

- **Client Environment**: Modern web browsers supporting HTML5, CSS3, ES6+, and the Fetch API (Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge).
- **Server Runtime**: Node.js active Long Term Support (LTS) runtime (`v20+` or `v24+`) executing native ECMAScript Modules (ESM).
- **Persistence Layer**: Embedded SQLite relational database using the built-in Node.js `node:sqlite` module with Write-Ahead Logging (`PRAGMA journal_mode = WAL`) and foreign key enforcement enabled.
- **External Network Dependencies**: Outbound HTTPS port 443 connectivity to the GitHub REST API (`https://api.github.com`) for telemetry synchronization.

2.5 Design

ProjNaN follows a modular 4-tier web architecture partitioned into a client presentation layer, an API gateway and security layer, a domain service layer, and a relational persistence layer.

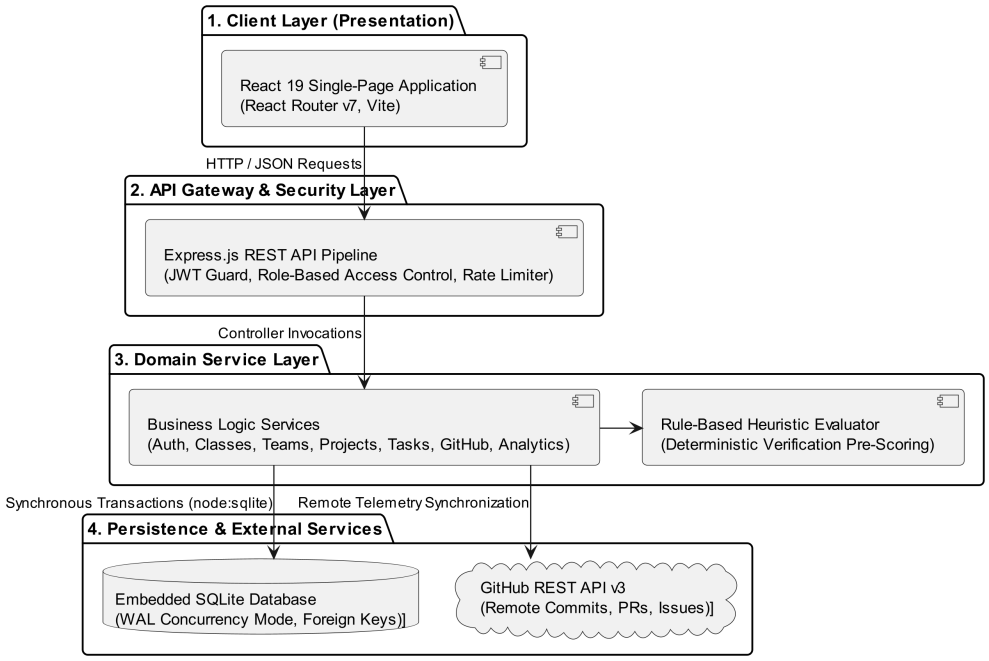


Figure 2.3: ProjNaN Simplified 4-Tier System Architecture

Figure 2.3 illustrates the system components:

- **Client Layer**: A responsive Single-Page Application (SPA) built with React 19, React Router v7, and Vite, featuring Kanban boards, telemetry tabs, and velocity graphs.
- **API Gateway Layer**: An Express.js REST API enforcing request sanitization, sliding-window rate limiting, stateless JWT authentication, and role-based access control.
- **Domain Service Layer**: Modular business logic services handling authentication, classes, teams, projects, tasks, telemetry, contributions, and gamification, integrated with the internal rule-based heuristic evaluator.
- **Persistence Layer**: Embedded SQLite storage operating in WAL mode to guarantee ACID transactional consistency, indexed queries, and cascade safety.

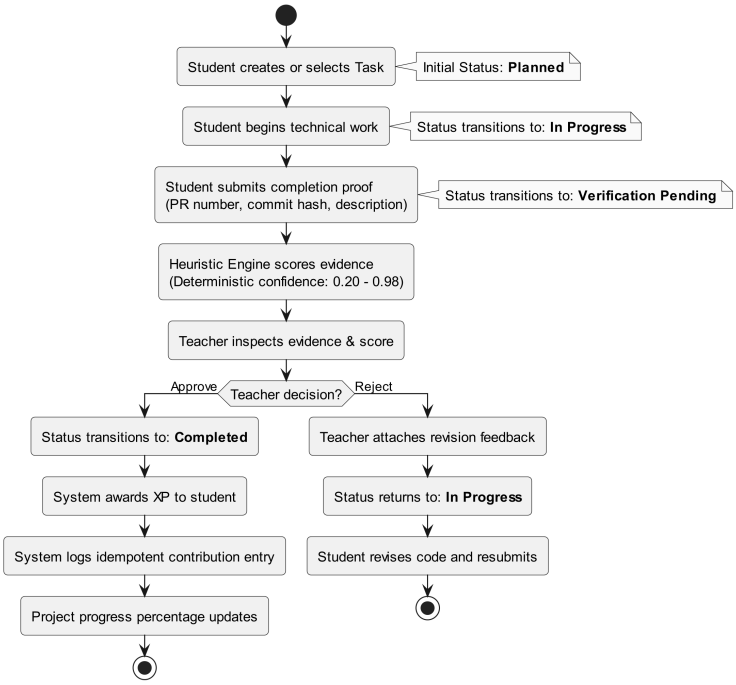


Figure 2.4: Task Verification and Lifecycle Activity Flow

Figure 2.4 details the task lifecycle activity flow. Tasks begin in the `Planned` state and move to `In Progress` when work starts. Submitting evidence transitions the task to `Verification Pending` and triggers the rule-based evaluator. The course instructor reviews the evidence and either approves the task (moving it to `Completed` and awarding XP) or rejects it with feedback (returning it to `In Progress` for student revision).

3 System Features

This chapter details the functional features of ProjNaN. All protected endpoints follow standard RESTful error conventions: unauthenticated requests return HTTP 401 Unauthorized, invalid input returns HTTP 400 Bad Request, unauthorized role actions return HTTP 403 Forbidden, and rate-limit violations return HTTP 429 Too Many Requests.

3.1 Description and Priority

The system capabilities are prioritized in Table 3.1.

Module ID	Feature Description	Priority	Primary Actor
MOD-01	User Authentication, Session, and RBAC	High	All Users
MOD-02	Academic Class Scoping and Team Formation	High	Teacher / Student
MOD-03	Project Workspaces and Milestone Tracking	High	Student / Teacher
MOD-04	Task Lifecycle (4-Stage State Machine)	High	Student / Teacher
MOD-05	Heuristic Evidence Evaluation Engine	High	System
MOD-06	Faculty Task Verification and Confirmation	High	Teacher
MOD-07	GitHub Telemetry Synchronization	Medium	Student / System
MOD-08	Idempotent Contribution Ledger & Analytics	High	System / All Users
MOD-09	Gamification (Weekly Arena & Leaderboard)	Medium	Student
MOD-10	Pedagogical Feedback (1–5 Star Reviews)	Medium	Teacher
MOD-11	Persistent Notifications & Project Channels	Low	All Users

Table 3.1: System Feature Priority Matrix

3.2 Functional Requirements

3.2.1 MOD-01: User Authentication, Session Management, and RBAC

Manages identity lifecycles, role-based authorization, email verification, and password recovery.

- **FR-1.1: Account Registration**: The system shall accept user registration with name, unique username, unique email address, password, and role (`'teacher'` or `'student'`). Passwords must contain at least 6 characters including uppercase, lowercase, and numeric digits, hashed using `bcryptjs` with 10 salt rounds.
- **FR-1.2: Email Verification Token**: Upon registration, the system shall generate a random 64-character hexadecimal verification token and initialize `email_verified = 0`.
- **FR-1.3: User Authentication**: The system shall validate login credentials against stored bcrypt hashes and issue an HMAC-SHA256 signed JWT containing `userId`, `email`, and `role`, valid for 7 days.

- **FR-1.4: Password Reset Lifecycle**: The system shall support a two-stage password recovery workflow. Requesting a reset generates a 64-hex token valid for 1 hour. Submitting a compliant new password updates the hash and invalidates the token.
- **FR-1.5: Role-Based Authorization Guard**: The API gateway shall intercept incoming requests using role middleware, rejecting unauthorized role access with HTTP 403 Forbidden.

3.2.2 MOD-02: Academic Class Scoping and Team Formation

Provides course scoping by enabling instructors to create classes and allowing students to form collaborative teams.

- **FR-2.1: Class Creation**: Instructors shall create academic classes with a course title and description. The system shall automatically generate a unique alphanumeric class join code (e.g., `CS101-ABCD`).
- **FR-2.2: Student Self-Enrollment**: Students shall join an academic class by submitting a valid class join code. The system validates the code and adds the student to the class roster.
- **FR-2.3: Class Roster Visibility**: Instructors shall be capable of viewing the complete roster of enrolled students, their enrollment dates, and assigned project teams.
- **FR-2.4: Team Formation**: Students enrolled within a class shall form collaborative teams by specifying a team name, designating the creator as initial team leader.

3.2.3 MOD-03: Project Workspaces and Milestone Tracking

Governs project definitions, progress calculations, and milestone deadlines.

- **FR-3.1: Project Workspace Instantiation**: Each team shall establish a project workspace categorized under their enrolled class, containing project title, description, goals, and topical tags.
- **FR-3.2: Milestone Definition**: Projects shall contain milestones with titles, descriptions, target due dates, and completion XP values (`xp_reward`, default 100 XP).
- **FR-3.3: Dynamic Progress Calculation**: The system shall dynamically compute overall project completion progress as an integer percentage:

$$\text{Progress} = \left(\frac{N_{\text{completed tasks}}}{N_{\text{total tasks}}} \right) \times 100 \tag{3.1}$$

  The calculated percentage updates synchronously upon any task status transition.

3.2.4 MOD-04: Task Management and 4-Stage State Lifecycle

Enforces a structured 4-stage state machine across units of project work.

- **FR-4.1: Task Creation**: Team members shall create tasks with attributes: `title`, `description`, `assigned_to_id`, `milestone_id`, `priority` (`'low'`, `'medium'`, `'high'`, `'urgent'`), `task_type` (`'daily'`, `'weekly'`, `'monthly'`), `deadline`, and `xp_value` (default 50 XP). Initial status is set to `'Planned'`.
- **FR-4.2: Work Inception**: When an assigned student begins implementation, task status transitions from `'Planned'` to `'In Progress'`.

- **FR-4.3: Evidence Submission Gate**: A task cannot transition directly from `'In Progress'` to `'Completed'`. The student must submit non-empty technical evidence text to trigger transition to `'Verification Pending'`.
- **FR-4.4: Task Assignment Notification**: Assigning a task to a team member shall automatically generate an in-app notification for that user.

3.2.5 MOD-05: Evidence Submission and Rule-Based Heuristic Evaluation

Analyzes student-submitted evidence using deterministic pattern matching and local repository cross-referencing.

- **FR-5.1: Evidence Text Validation**: Submitting completion evidence requires non-empty text. Empty submissions return HTTP 400 Bad Request.
- **FR-5.2: Heuristic Confidence Evaluation**: The internal engine evaluates evidence text through a deterministic rule-based formula:

$$S_{\text{confidence}} = \min\left(0.98,\ \max\left(0.20,\ S_{\text{base}} + \delta_{\text{len}} + \delta_{\text{PR}} + \delta_{\text{commit}} + \delta_{\text{test}}\right)\right) \tag{3.2}$$

  where $S_{\text{base}} = 0.50$, length bonus $\delta_{\text{len}} = 0.15$ (> 30 characters), PR bonus $\delta_{\text{PR}} = +0.25$ (matched in `github_activities`) or $+0.10$ (regex pattern cited), commit SHA bonus $\delta_{\text{commit}} = +0.25$ (matched in database) or $+0.10$ (regex pattern cited), and test outcome bonus $\delta_{\text{test}} = +0.10$. Final score is clamped between 0.20 and 0.98 (typically 0.50 to 0.90).
- **FR-5.3: Heuristic Explanation Compilation**: The system shall construct human-readable explanatory tags detailing the rationale for the calculated score, formatted as `AI Verification (<Score>%): <reasons>` (for example: "`AI Verification (90%):` Detailed descriptive summary provided; Verified pull request #12 linked in repository").
- **FR-5.4: Verification Persistence**: The system shall persist the evaluation record with status `'suggested'` linked to the task.

3.2.6 MOD-06: Faculty Task Verification and Confirmation

Empowers course instructors to make authoritative verification decisions on submitted tasks.

- **FR-6.1: Role Guard Enforcement**: Only authenticated users with `role = 'teacher'` are permitted to invoke the verification endpoint. Student attempts return HTTP 403 Forbidden.
- **FR-6.2: Approval Decision Execution**: When an instructor approves a task:
  1. Task status updates to `'Completed'` and `verification_status = 'verified'`.
  2. Verification record transitions to `'confirmed'`.
  3. The assigned student's profile is credited with `xp = xp + task.xp_value` and contribution points.
  4. An entry is inserted into the idempotent `contributions` ledger with `external_id = 'task:<id>'`.
  5. Overall project progress percentage is recalculated.
  6. A persistent notification is delivered to the student.
- **FR-6.3: Rejection Decision Execution**: When an instructor rejects a task:
  1. Task status returns to `'In Progress'` and its `verification_status` updates to `'rejected'`.

2. Verification record updates to `'rejected'`, appending the teacher's feedback notes.
3. No XP or contribution points are awarded.
4. A notification detailing required revisions is dispatched to the student.

3.2.7 MOD-07: External GitHub Telemetry Synchronization

Synchronizes remote commits, pull requests, and issues from the GitHub REST API v3 into local relational storage.

- **FR-7.1: Repository Connection**: A project workspace shall connect to a remote GitHub repository by specifying a valid repository URL and an optional Personal Access Token (PAT).
- **FR-7.2: Remote Endpoint Ingestion**: Triggering synchronization invokes the GitHub REST API v3 to fetch recent commits, pull requests, and issues.
- **FR-7.3: Activity Ingestion and Deduplication**: Fetched items are normalized and inserted into `github_activities`, recording commit SHAs, PR numbers, author logins, and timestamps.
- **FR-7.4: Network Fault Resilience**: In the event of network timeouts, DNS resolution failures, or GitHub rate limiting, the synchronization pipeline shall catch exceptions gracefully, set repository sync status to `'error'`, and prevent application crashes.

3.2.8 MOD-08: Idempotent Contribution Ledger and Team Analytics

Maintains an idempotent activity ledger that prevents double-counting across repeated sync passes and computes team velocity and relative effort distribution.

- **FR-8.1: Idempotency Key Enforcement**: Every contribution record requires a deterministic external reference key (`external_id`), such as `commit:<sha>`, `pr:<repo>:<num>`, or `task:<taskId>`. If an entry with an identical `external_id` exists, insertion is safely skipped without duplicate XP attribution.
- **FR-8.2: Author Matching Logic**: When repository activity is ingested, the system attempts to match GitHub author logins or commit author names against registered project team members. Unmatched commits remain stored in `github_activities` but do not enter the contribution ledger.
- **FR-8.3: Relative Effort Share Calculation**: The analytics engine calculates each team member's relative contribution share (P_i):

$$P_i = \left(\frac{C_i}{\sum_{k=1}^{N} C_k} \right) \times 100 \tag{3.3}$$

  where C_i represents member i's total verified contribution count and N is the number of active team members.
- **FR-8.4: 14-Day Velocity Sparkline**: The system aggregates verified contributions across a rolling 14-day window to compute daily activity volume and XP velocity curves for frontend visualization.

3.2.9 MOD-09: Gamification, Weekly Arena, and Leaderboard

Incentivizes technical engagement through coding challenges, achievement badges, and XP-driven student ranks.

- **FR-9.1: Arena Challenge Catalog**: The platform hosts coding challenges categorized by difficulty (`'Easy'`, `'Medium'`, `'Hard'`) with assigned XP rewards (e.g., 150 XP).
- **FR-9.2: Challenge Submission**: Students accept and submit challenge solutions alongside verification evidence, earning XP upon acceptance.
- **FR-9.3: Gamified Rank Tiers**: Students progress through defined achievement ranks based on cumulative XP:
  - **Novice**: $0 - 249$ XP
  - **Explorer**: $250 - 499$ XP
  - **Contributor**: $500 - 999$ XP
  - **Expert**: $1000 - 1999$ XP
  - **Master**: ≥ 2000 XP
- **FR-9.4: Class and Global Leaderboard**: The system renders ranked leaderboards displaying total XP, earned badges, and completed challenges.

3.2.10 MOD-10: Pedagogical Feedback and 1–5 Star Reviews

Enables instructors and peers to attach qualitative feedback and standardized ratings to student projects.

- **FR-10.1: Instructor Project Review**: Faculty mentors shall submit structured evaluations containing a 1–5 star integer rating and qualitative commentary regarding architecture, code quality, and collaboration.
- **FR-10.2: Peer Feedback**: Team members shall provide constructive feedback on peer contributions.
- **FR-10.3: Review Notifications**: Submitting feedback shall generate an immediate notification for all members of the target project team.

3.2.11 MOD-11: Persistent Notifications and Project Channel Discussion

Provides asynchronous in-app notifications and project-scoped discussion channels.

- **FR-11.1: Event-Driven Alerts**: The system generates persistent notifications for task assignments, verification approvals/rejections, milestone deadlines, and feedback submissions.
- **FR-11.2: Read State Management**: Unread alerts are flagged with an unread badge and can be marked as read individually or in bulk.
- **FR-11.3: Project Discussion Channels**: Each project workspace contains dedicated chat channels (e.g., `#general`, `#dev`) for team communication.

4 Other Nonfunctional Requirements

4.1 Performance Requirements

- **NFR-1.1: Database Query Latency (Design Target)**: Local SQLite queries for indexed primary keys (user lookups, task status queries) are targeted to execute in under 10 milliseconds under nominal laboratory workloads.
- **NFR-1.2: API Endpoint Response Time (Design Target)**: Standard CRUD endpoints are targeted to return responses within 200 milliseconds under nominal concurrent student usage.
- **NFR-1.3: Telemetry Synchronization Duration (Design Target)**: Repository synchronization over a stable Internet connection is targeted to ingest recent commits and pull requests within 5 seconds.
- **NFR-1.4: Frontend Bundle Footprint (Verified Target)**: Production Single-Page Application assets are targeted to remain under 450 kB uncompressed ($\leq$ 125 kB gzip-compressed) to ensure rapid loading on campus networks. An empirical production build (`npm run build` via Vite) was verified: the compiled JavaScript client bundle is exactly 412.44 kB (113.60 kB gzip-compressed), and the compiled stylesheet bundle is 29.64 kB (6.30 kB gzip-compressed), fully satisfying the performance target.

4.2 Security Requirements

- **NFR-2.1: Password Hashing**: Raw passwords shall never be stored in plain text. Hashing must employ `bcryptjs` with a minimum of 10 salt rounds.
- **NFR-2.2: Cross-Site Scripting (XSS) Sanitization**: The application pipeline incorporates recursive sanitization middleware that strips script injection vectors from all incoming request bodies.
- **NFR-2.3: Authentication Rate Limiting**: To mitigate brute-force credential guessing, an in-memory sliding-window rate limiter restricts authentication attempts to a maximum of 15 requests per 15-minute window per IP address, responding with HTTP 429 upon breach.
- **NFR-2.4: RBAC Enforcement**: Crucial administrative operations (approving task verifications, creating classes) strictly enforce teacher authorization, guaranteeing that student attempts return HTTP 403 Forbidden.
- **NFR-2.5: Token Expiration and Isolation**: Password reset tokens are constrained to a strict 1-hour expiration window; JWT session tokens expire after 7 days.
- **NFR-2.6: Transactional Atomicity**: Multi-table state mutations (e.g., confirming task verification, updating task status, crediting user XP, and logging the contribution ledger) execute within atomic database transactions. In the event of an unhandled runtime error, changes automatically roll back. The SQLite engine operates in Write-Ahead Logging (WAL) mode, providing crash resilience during unexpected process termination.

4.3 Software Quality Attributes

- **Reliability**: The platform delivers high availability through graceful fault tolerance. Network errors during GitHub synchronization are caught without crashing the application server.
- **Maintainability**: The backend follows a clean, modular service-oriented architecture (controllers, services, middleware, database models) separating concerns.
- **Usability**: The interface minimizes cognitive load by surfacing heuristic confidence tags and color-coded statuses, allowing instructors to evaluate student tasks quickly.
- **Testability**: The functional stability and security guards of ProjNaN are verified through 52 automated tests executed by the Node.js native test runner (`node:test`). All 52 tests pass with 100% success across authentication (16 tests), core workflow (23 tests), gamification (6 tests), and security guards (7 tests).

4.4 Business Rules

- **BR-1: Verified Task Completion**: A task cannot transition to `'Completed'` unless an authenticated course instructor (`teacher`) explicitly confirms verification. Student self-completion is prevented, and no unverified administrative override exists in the system.
- **BR-2: Single Active Repository Link**: Each project workspace can link to at most one remote GitHub repository at any given time (`UNIQUE(project_id)` constraint in `repositories`).
- **BR-3: Idempotent Ledger Integrity**: Contribution XP can be granted for a specific Git commit or completed task at most once. Idempotency is enforced by the `external_id` index in the `contributions` table.
- **BR-4: Class Join Code Uniqueness**: Class codes must remain globally unique across the institution to prevent ambiguous course enrollment.

5 Other Requirements

5.1 Maintenance Requirements

As software engineering pedagogical practices evolve, ProjNaN requires ongoing maintenance:

- **Schema Evolution**: Future relational migrations must execute via transactional migration scripts, ensuring backward compatibility.
- **Database Archival**: At the conclusion of each academic semester, completed classes and associated telemetry can be archived to prevent database file bloat.

5.2 Future Roadmap

The following architectural extensions represent planned future enhancements:

1. **LLM Semantic Code Diff Review**: Integrating Large Language Model (LLM) agents to analyze Git code diffs semantically for architectural adherence and code quality.
2. **Full-Duplex WebSocket Infrastructure**: Transitioning project channel chat from HTTP REST polling to persistent WebSockets for real-time messaging.
3. **Cross-Project Plagiarism Detection**: Integrating source-code similarity algorithms (e.g., automated MOSS API adapters) to detect unauthorized code copying between team repositories.